

Documentação - Release Dados Abertos

Utilidade:	Automatização do processo dos Dados Abertos
Criado em:	31/07/2024
Última Atualização:	31/07/2024
Descrição:	Código Python, utilizando Jupyter Notebook e outras bibliotecas para a automatização do processo de análise de dados dos Releases de dados Abertos.
Responsável pela Atualização:	Pietro Zamperlini Bontempi
Fonte:	Times New Roman

Índice

1. Resumo Geral	1
2. Índice	2
3. Objetivo do Código	3
4. Ferramentas Necessárias	4
5. Entendendo o Código	12
6. Observações.....	30
7. Erros e Possíveis Soluções.....	32
8. Links Úteis e Bibliografia.....	36
9. Glossário.....	37

Objetivo do código

Este projeto tem como objetivo principal automatizar o processo de releases de dados abertos, proporcionando uma solução eficiente e robusta para a extração, transformação e disponibilização de dados. Em um cenário onde a gestão e o compartilhamento de informações se tornam cada vez mais cruciais, a capacidade de manipular grandes volumes de dados de forma precisa e ágil é fundamental.

A primeira etapa do processo consiste na leitura de dados a partir de uma planilha CSV, uma das formas mais comuns de armazenamento de dados estruturados. Ao transformar esses dados em data frames, utilizamos uma estrutura flexível e poderosa que permite uma ampla gama de operações de manipulação e análise. Esta transformação é essencial para garantir que os dados sejam tratados de maneira organizada e que estejam prontos para etapas subsequentes de processamento.

Uma vez que os dados estejam em data frames, diversas operações de manipulação podem ser realizadas. Isso inclui limpeza de dados, filtragem, agregação, e outras transformações necessárias para que os dados estejam no formato ideal para a sua análise e utilização. Esta etapa é crucial para assegurar a integridade e a qualidade dos dados, eliminando inconsistências e preparando-os para uma análise detalhada.

O ponto culminante do projeto é a geração de um arquivo Excel contendo os dados extraídos e manipulados. O formato Excel é amplamente utilizado e oferece uma interface amigável para a visualização e compartilhamento de dados. A geração automática deste arquivo não só economiza tempo, mas também minimiza a possibilidade de erros humanos, garantindo que os dados estejam prontos para serem utilizados em decisões estratégicas, relatórios ou outras aplicações.

Em resumo, este projeto visa simplificar e otimizar o processo de releases de dados abertos, proporcionando uma ferramenta que transforma dados brutos em informações valiosas de maneira automática e eficiente. Através da automação, esperamos não só aumentar a produtividade, mas também melhorar a precisão e a acessibilidade dos dados, promovendo uma cultura de transparência e eficiência no manejo de informações.

Ferramentas Necessárias

Para prosseguir com o projeto, serão necessárias as instalações de algumas ferramentas e extensões para que o código possa rodar em seu computador ou notebook. Essas ferramentas são:

- Visual Studio Code
 - Jupyter Notebook
 - Python (Extensão Visual Studio Code)
- Python
 - Biblioteca Pandas
 - Biblioteca XLSxWriter

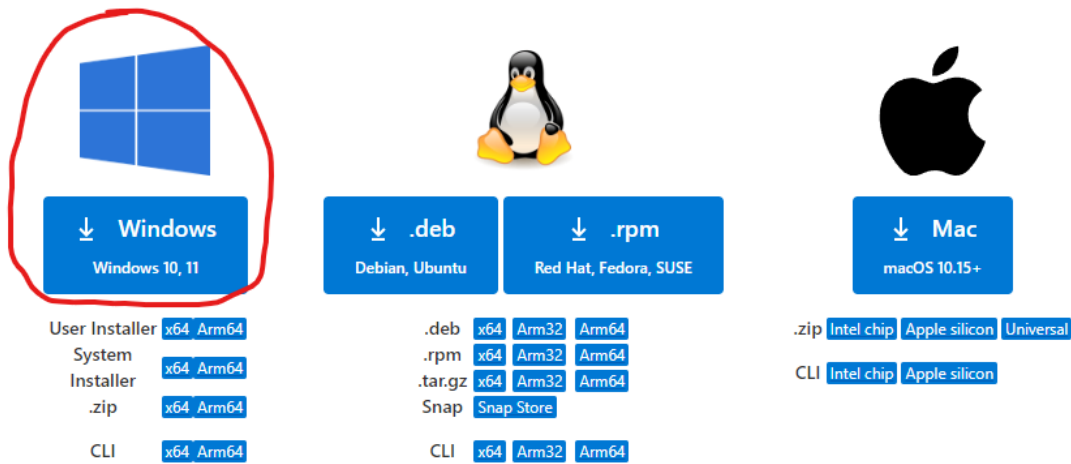
Instalação Visual Studio Code

Primeiro Passo é acessar o link a seguir: <https://code.visualstudio.com/download>

Após acessar o link, aparecerá para o usuário selecionar o Sistema Operacional que deve ser utilizado para a instalação, seja ele Windows, Linux ou Mac. Como as máquinas da prefeitura em sua maioria são Windows, irei realizar o processo selecionando a opção de Windows 10 ou 11:

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.

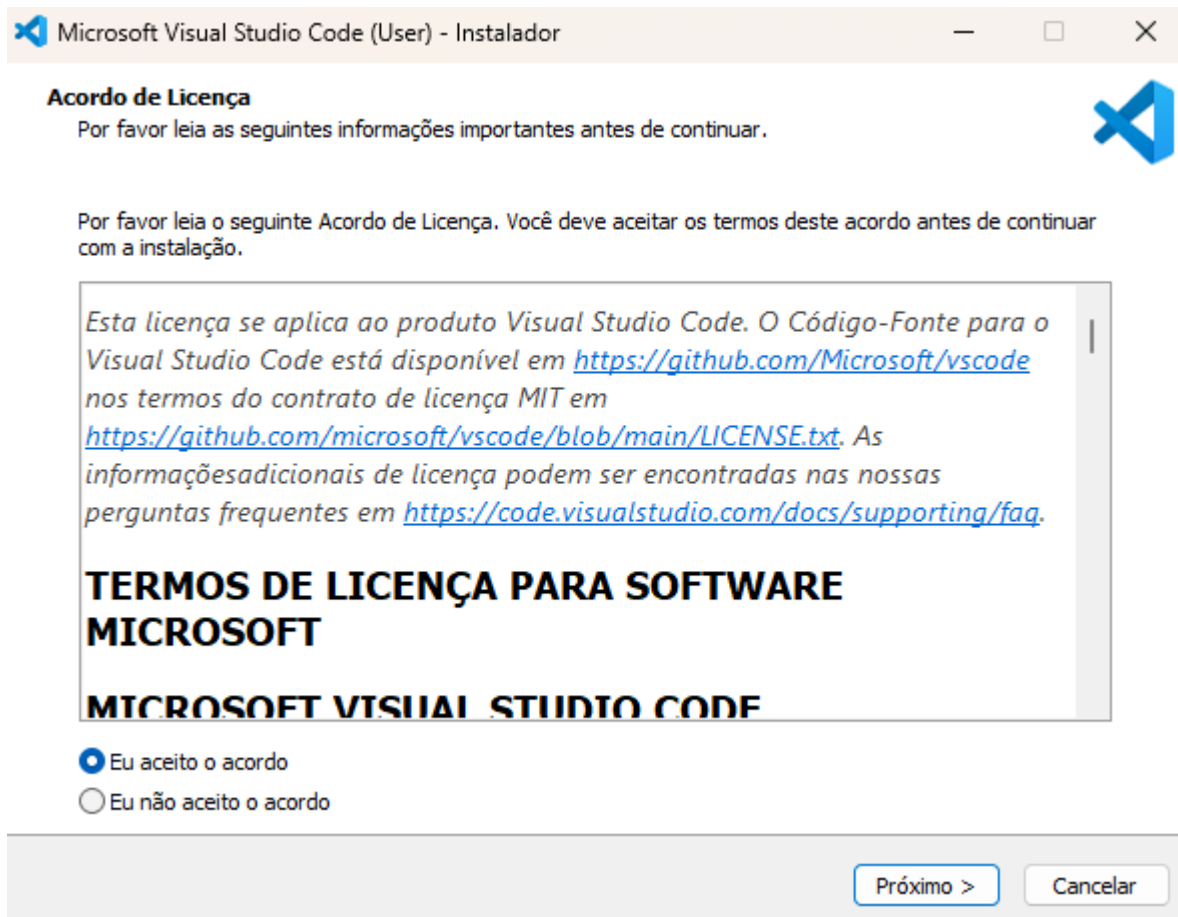


Quando selecionado esta opção, automaticamente o navegador irá baixar o arquivo de instalação do Visual Studio Code.

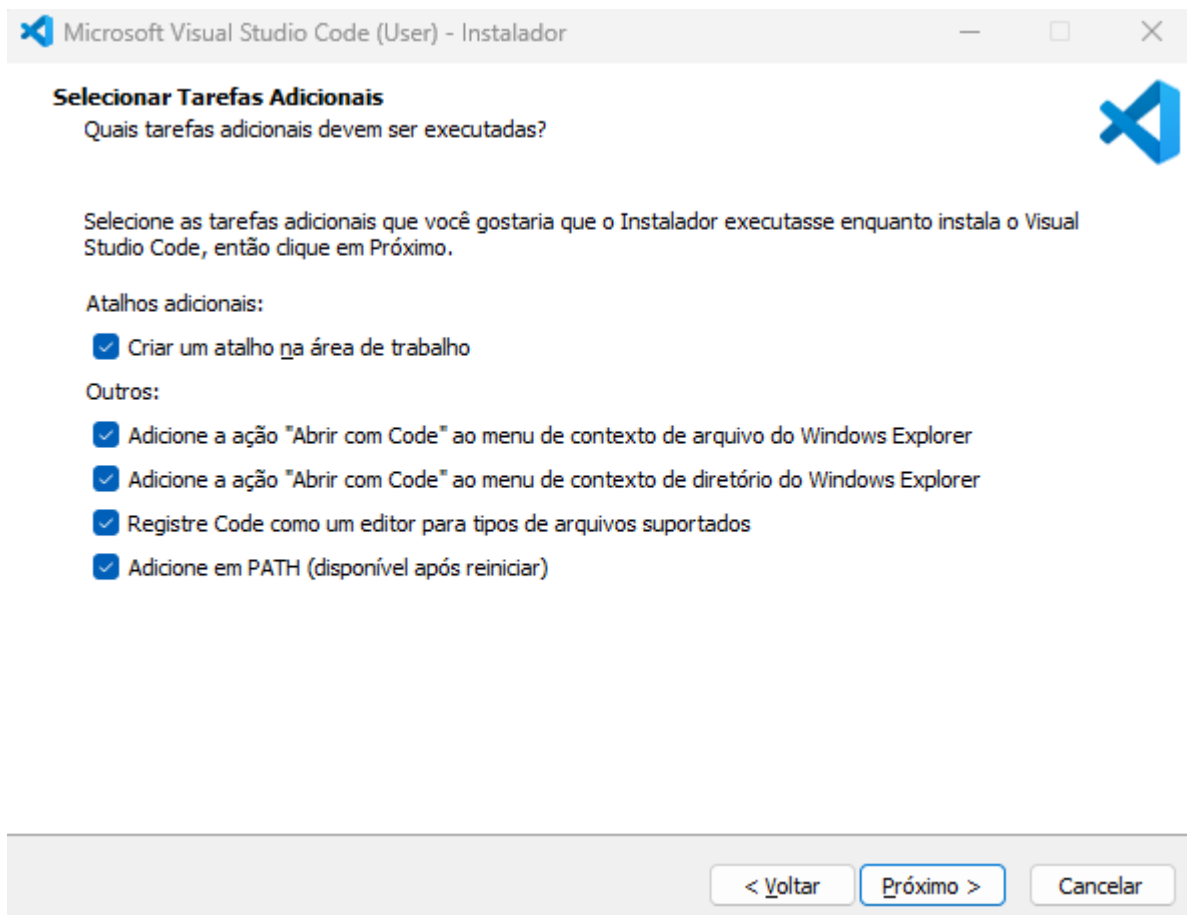
 VSCodeUserSetup-x64-1.89.1.exe

Esse arquivo irá aparecer na sua pasta de Downloads, ou em outra pasta do seu computador. Quando achar o arquivo, dê um duplo clique nele, e logo em seguida ele abrirá a tela de instalação. Caso o sistema solicite permissão de administrador, clique em sim.

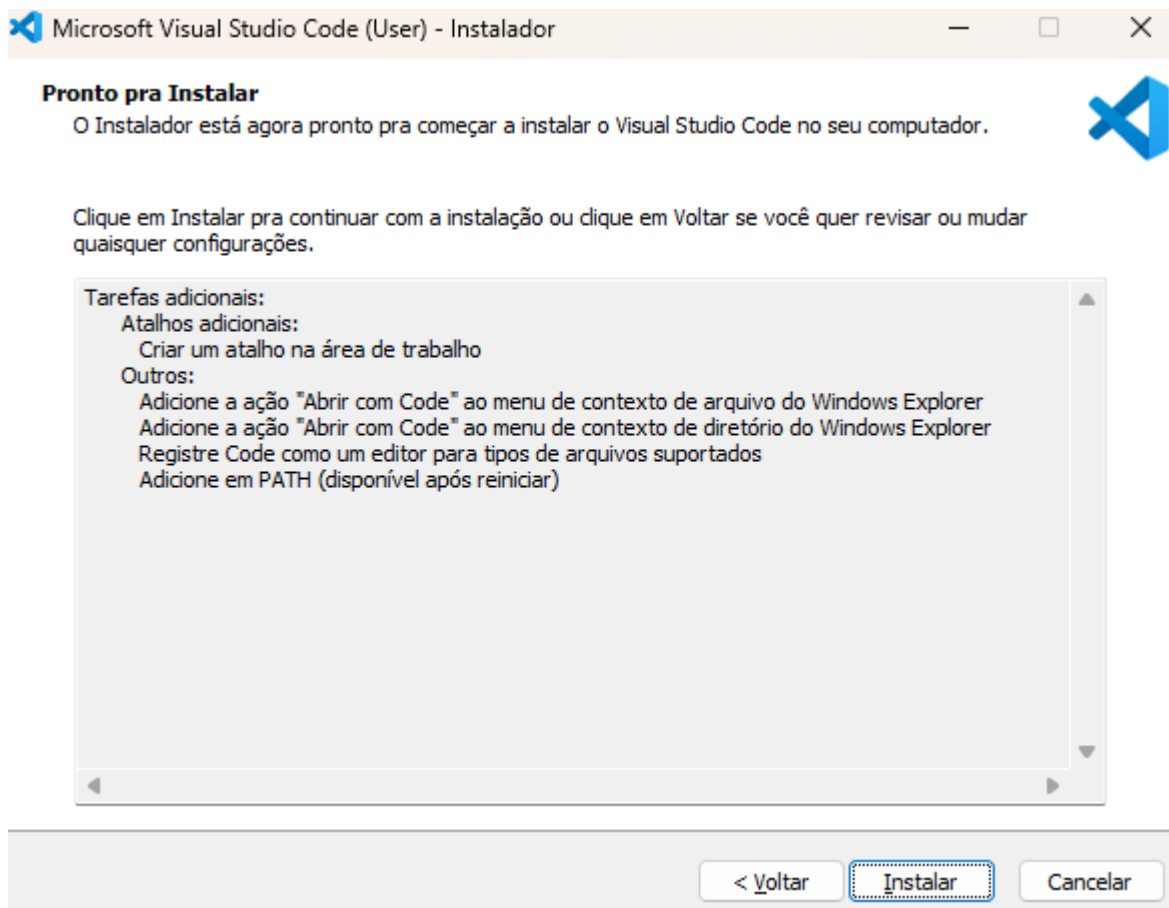
Na primeira tela, você verá o acordo de licença e termos de uso do software, clique em “Eu aceito o acordo” e logo em seguida clique em próximo.



Na opção seguinte, deixe tudo selecionado, caso não queira criar um atalho na área de trabalho, desmarque a opção “Criar um atalho na área de trabalho”, caso contrário, deixe tudo selecionado e clique em próximo.



Após isso, clique em instalar e você terá o software pronto para uso.

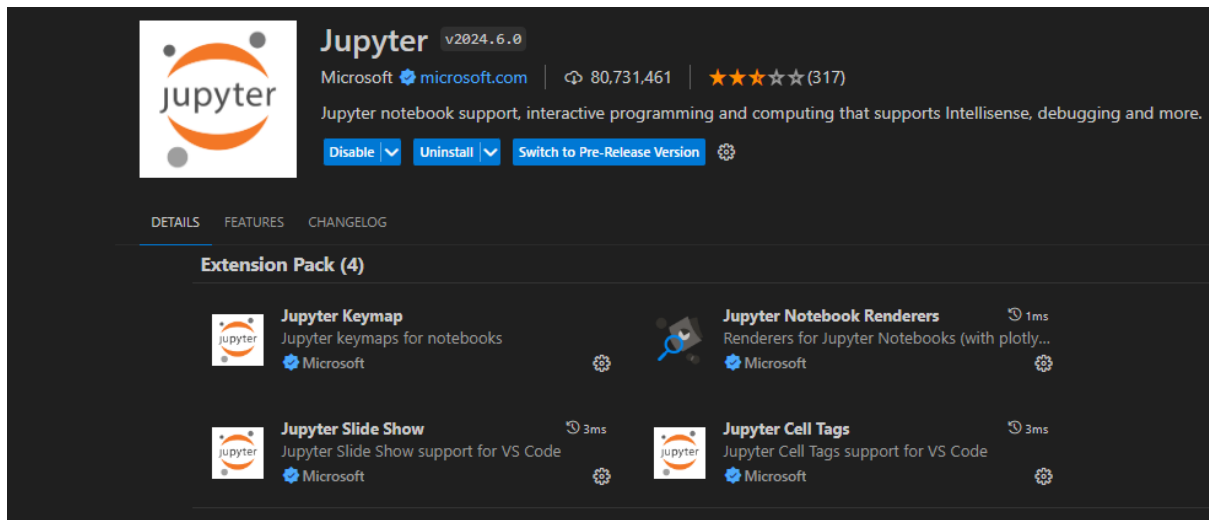


Instalação das Extensões Jupyter Notebook e Python

Com o Visual Studio Code aberto, selecione a opção “Extensions” na barra lateral esquerda, ou use o atalho Ctrl+Shift+X para acessar.

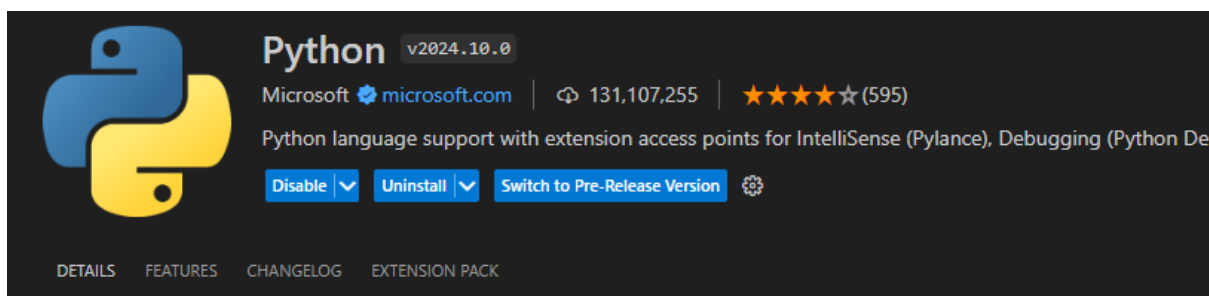


Na barra de pesquisa digite Jupyter, irão aparecer algumas opções, clique na primeira opção e logo em seguida clique em instalar.



Como parte do processo de instalação, ele instalará mais 4 bibliotecas junto, as quais estão na imagem acima. Não é necessário nenhum tipo de configuração do aplicativo por enquanto.

Agora teremos que fazer a instalação do python no VSCODE para utilizarmos ele. Ainda na barra de pesquisa de extensões digite python, clique no primeiro item e faça a instalação dele.



Com essas duas extensões instaladas, podemos prosseguir para os próximos passos. Lembrando, o VSCODE possui uma extensa variedade de extensões dos mais diversos tipos, fique à vontade para explorar essas extensões, mas para o projeto, apenas essas duas são cruciais para o desenvolvimento.

Instalando o Python 3.12.4

No atual momento desta documentação, a versão que está sendo utilizada do Python é a versão 3.12.4, caso esteja em uma versão superior, pode utilizar esse mesmo passo a passo para a instalação.

Para começar acesse este link: <https://www.python.org/downloads/>

Quando acessar, faça o download da versão 3.12.4 do python, ou versão superior.

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.12.4	June 6, 2024	Download	Release Notes
Python 3.12.3	April 9, 2024	Download	Release Notes
Python 3.11.9	April 2, 2024	Download	Release Notes
Python 3.10.14	March 19, 2024	Download	Release Notes
Python 3.9.19	March 19, 2024	Download	Release Notes
Python 3.8.19	March 19, 2024	Download	Release Notes
Python 3.11.8	Feb. 6, 2024	Download	Release Notes
Python 3.11.3	Feb. 6, 2024	Download	Release Notes

[View older releases](#)

Depois, selecione a opção “Windows Installer (64-bit)” ou a versão correspondente do seu Windows para baixar o instalador do Python.

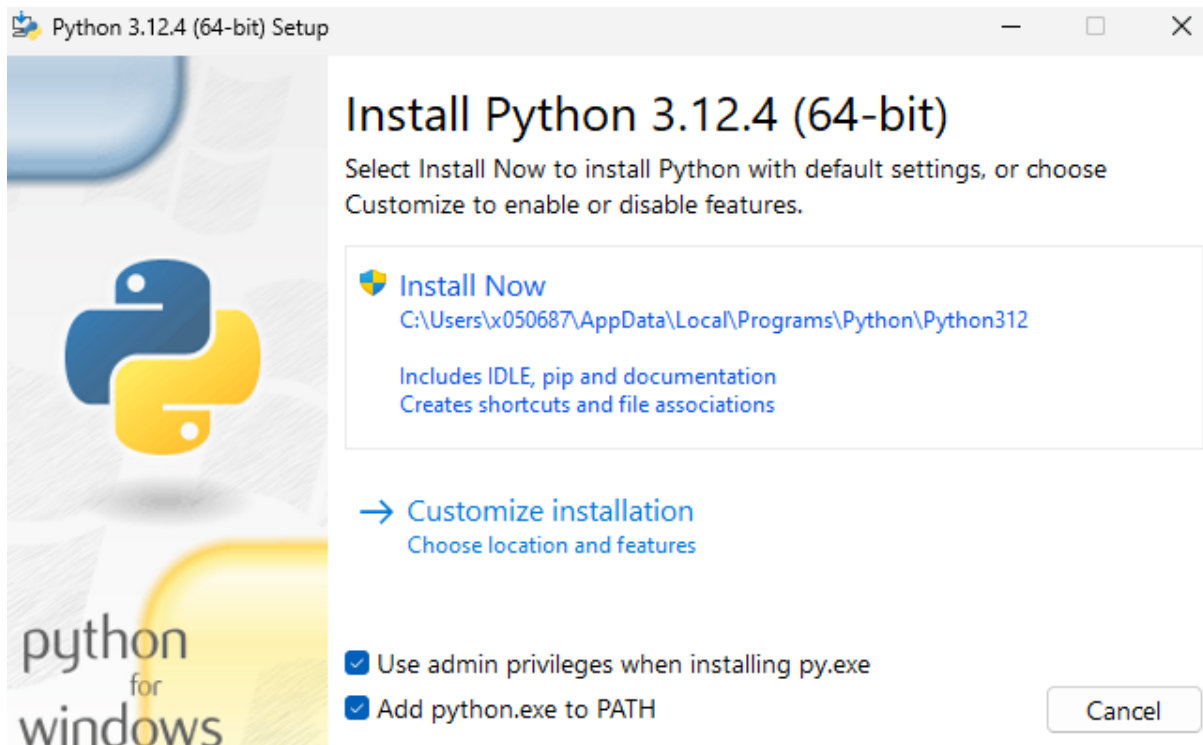
Files

Version	Operating System	Description	MD5 Sum	File Size	GPG	Sigstore	SBOM
Gzipped source tarball	Source release		ead819dab6d165937138daa9e51ccb54	26.0 MB	SIG	.sigstore	SPDX
XZ compressed source tarball	Source release		d68f25193eec491eb54bc2ea664a05bd	19.7 MB	SIG	.sigstore	SPDX
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	b6de6aea008605f5d4096014c2ad3c43	44.0 MB	SIG	.sigstore	
Windows installer (64-bit)	Windows	Recommended	f3df1be26cc7cbd8252ab5632b62d740	25.5 MB	SIG	.sigstore	SPDX
Windows installer (32-bit)	Windows		d9c98b529889aba04ca5ec1c6b5f986f	24.3 MB	SIG	.sigstore	SPDX
Windows installer (ARM64)	Windows	Experimental	f3c2064f11c5f4eee475928a0fc62199	24.8 MB	SIG	.sigstore	SPDX
Windows embeddable package (64-bit)	Windows		8db759b337ac4f6966f52b3662c05dd7	10.6 MB	SIG	.sigstore	SPDX
Windows embeddable package (32-bit)	Windows		19691145551a41114b32a556bb2bcb89	9.4 MB	SIG	.sigstore	SPDX
Windows embeddable package (ARM64)	Windows		0a863fd2485b3057a2eea108f1252160	9.8 MB	SIG	.sigstore	SPDX

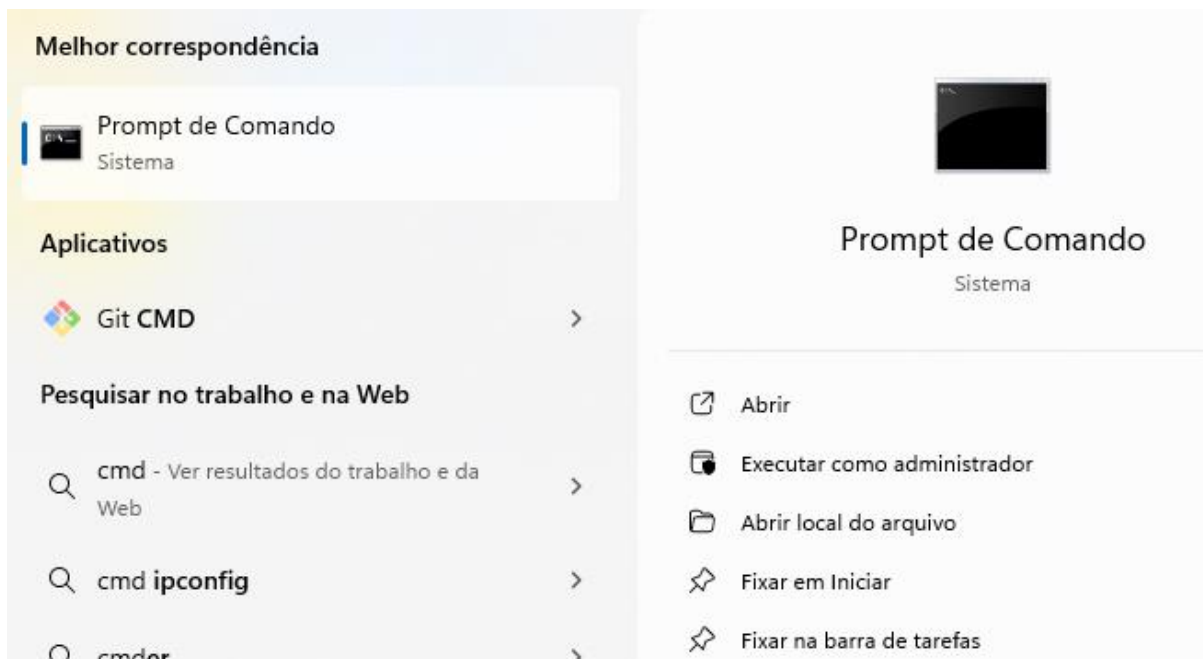
Ao clicar na opção escolhida, ele começará a baixar o instalador. Na sua pasta Downloads, ou a pasta que você escolheu, dê um duplo clique no instalador para começar o processo de instalação



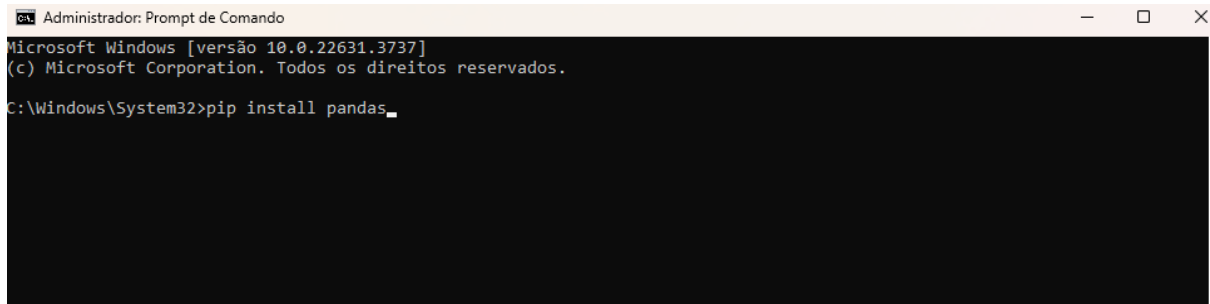
O processo de instalação é bem simples, você irá selecionar as caixas “Use admin privileges when installing py.exe” e “Add python.exe to PATH”, após isso clique em “Install Now” para concluir o processo de instalação.



Após a conclusão da instalação ainda precisamos configurar algumas coisas. Na barra de pesquisa, digite CMD, clique com o botão direito e execute como administrador.

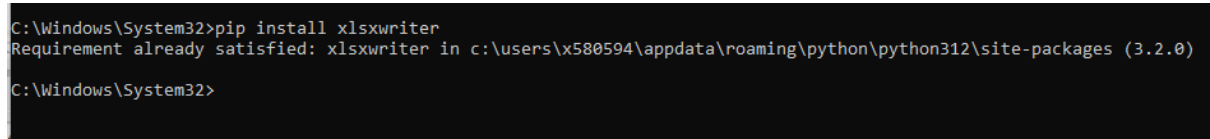


Com isso feito, vamos começar a instalação das nossas bibliotecas. No CMD digite “pip install pandas”, para instalarmos a biblioteca pandas.



```
Administrador: Prompt de Comando
Microsoft Windows [versão 10.0.22631.3737]
(c) Microsoft Corporation. Todos os direitos reservados.
C:\Windows\System32>pip install pandas_
```

Após instalar a biblioteca pandas, temos que instalar a xlswriter, digite na linha de comando, “pip install xlswriter”, para instalar a biblioteca.



```
C:\Windows\System32>pip install xlswriter
Requirement already satisfied: xlswriter in c:\users\x580594\appdata\roaming\python\python312\site-packages (3.2.0)
C:\Windows\System32>
```

Com todas essas bibliotecas já instaladas, agora podemos começar a entender e trabalhar com o código

Entendendo o Código

Sobre o Código (Informações do Arquivo)

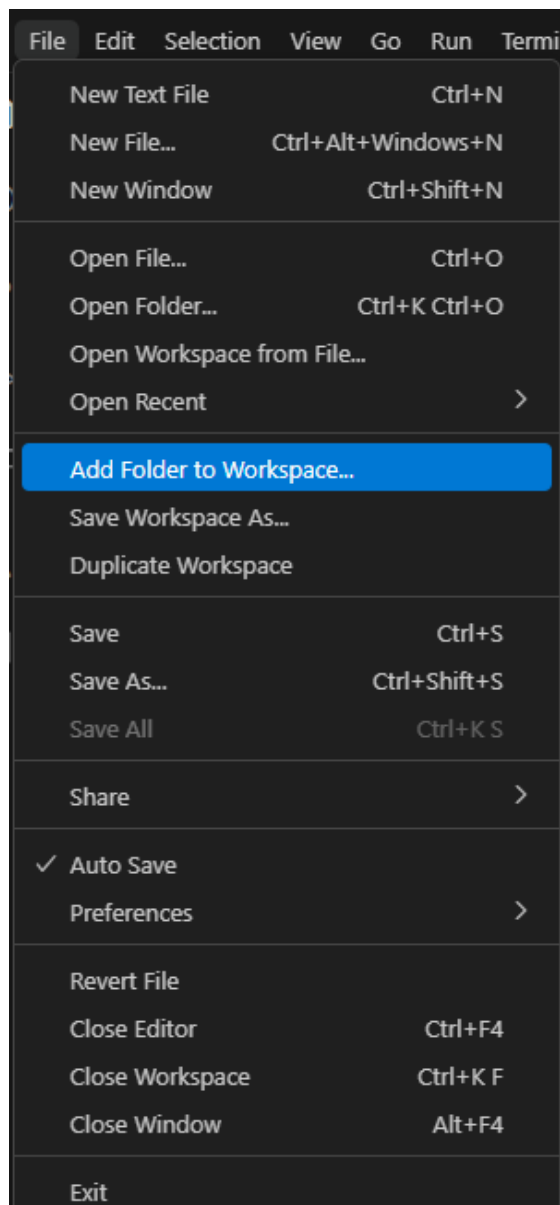
O arquivo do código tem uma extensão .ipynb, ou seja, um Arquivo Fonte Jupyter Notebook. Todos os códigos inseridos nele são feitos na linguagem de programação Python 3.12.4, mas ele possui compatibilidade com versões superiores futuras e com algumas versões mais antigas. O código é feito para analisar as planilhas de dados abertos que podem ser encontradas no link: http://dados.prefeitura.sp.gov.br/pt_PT/dataset/dados-do-sp156 .

O código analisa **SOMENTE** planilhas .csv , não tendo suporte para outros tipos de planilha como .xlsx. Como dito anteriormente, o objetivo é facilitar a leitura e manipulação de dados, além de facilitar o processo de montagem do Release.

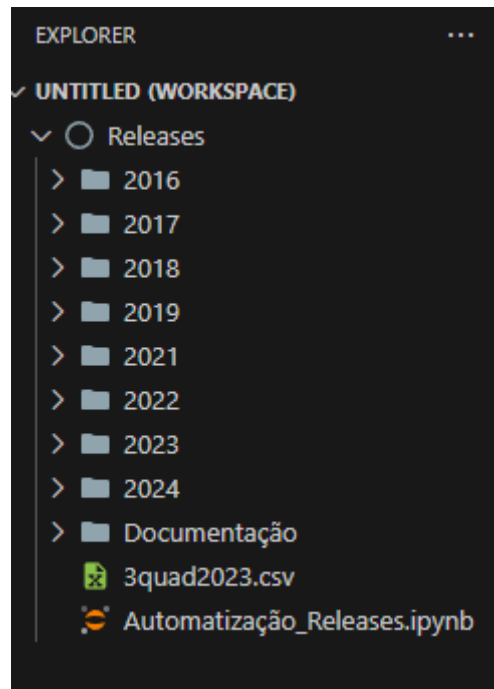
Analizando as Células do Código

Agora iremos analisar linha a linha do código para melhor compreensão da sua funcionalidade.

O primeiro passo é adicionar a pasta em que está o código no VSCODE, para isso, clique em File>Add Folder to Workspace e depois selecione a pasta em que está o código. Neste caso exemplo, o código está em uma pasta chamada Releases, dentro da Área de Trabalho



O segundo passo é realizar a importação do arquivo .csv, que pode ser retirado do link dos dados abertos comentado acima. O arquivo deve estar na mesma pasta que o código, para que ele possa ser acessado e ter as informações extraídas. Esse arquivo csv baixado pode ser renomeado da maneira que você preferir, como exemplo eu renomeei o arquivo para 3quad2023.csv, pois iremos fazer a análise do 3º quadrimestre de 2023 nesse caso.



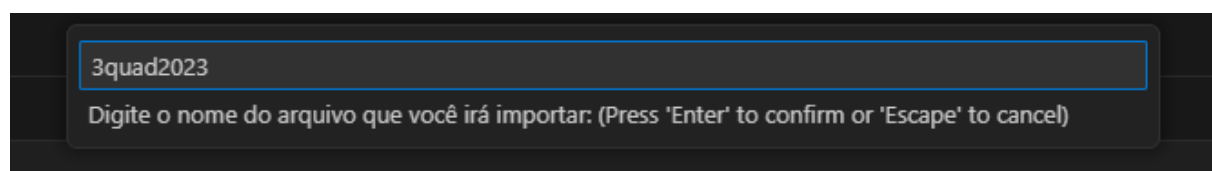
Com o arquivo na mesma pasta, podemos começar a fazer a análise do nosso código e compreender suas funções.

Definição de Variáveis fixas e importação do arquivo .csv

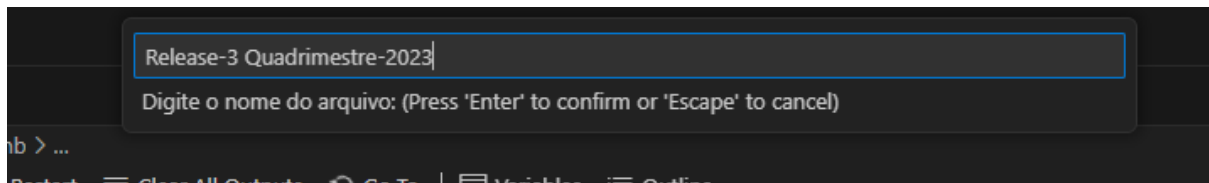
```
importacao = input("Digite o nome do arquivo que você irá importar:") + '.csv' #Definindo o nome do arquivo que estamos importando para o código
nome_arquivo = input('Digite o nome do arquivo: ') + '.xlsx' #Definindo o nome do arquivo de saída
ano_vigente = input("Digite o ano que está sendo trabalhado:") #Definindo o ano do Arquivo importado
periodo_ano = input("Digite o período do ano do arquivo:")#Ex: Primeiro trimestre, Segundo Semestre, etc.
#Definindo o período do ano do arquivo importando
```

Nessa primeira parte do código, definimos algumas variáveis fixas, que não irão mudar e serão chamadas durante todo o nosso código. Para rodar o código, clique no botão Play ao lado da célula.

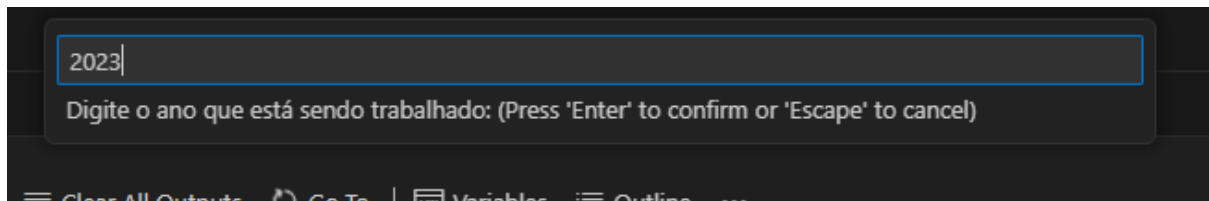
Na primeira linha, definimos a variável importação. Quando executado o código, pedirá ao usuário que digitemos o nome do arquivo que estamos importando, é **OBRIGATÓRIO** que o nome digitado seja o mesmo nome do arquivo .csv baixado. Para facilitar, foi feita uma condição no código que exclui a necessidade de digitar o nome do arquivo + .csv, sendo assim, só é necessário que coloque o nome do arquivo mesmo.



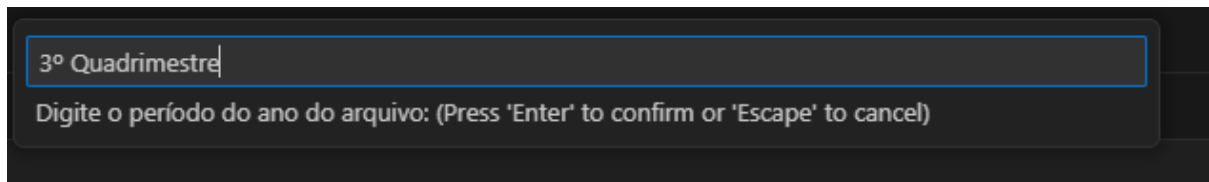
Após isso, você irá apertar enter, o que te levará para a segunda linha do código. Nesta segunda linha, digitaremos o nome do arquivo de saída, ou seja, o nome do Excel que será gerado. Para isso digite no campo o nome que deseja para o arquivo de saída.



Logo em seguida tecle novamente em Enter e iremos para a segunda linha de código. Nesta segunda linha, definiremos o ano vigente do arquivo, ou seja, o ano que estamos analisando. Para isso, basta digitar o ano analisado no campo de input e teclar enter novamente



Logo em seguida, digitaremos o período do ano que estamos analisando, ou seja, o trimestre, quadrimestre, bimestre ou semestre que estamos analisando, digite isso no campo input.



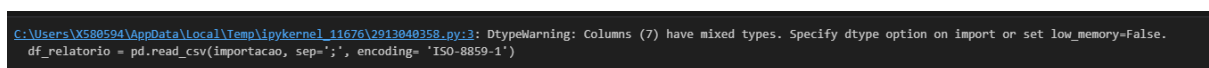
Com essa etapa finalizada, iremos para a segunda Célula do código.



Nessa parte do código, faremos a importação da biblioteca pandas, previamente instalada para nosso Script e transformaremos nosso arquivo CSV em um DataFrame.

Na segunda linha, utilizamos a variável importação, definida na célula anterior, colocamos como o CSV está separado, neste caso por “;” mas poderia ser por “,” também. E definimos também a linguagem de codificação dos caracteres como ISO-8859-1, uma das mais usuais.

A saída esperada é essa:



Finalizando está célula, vamos para a terceira célula do nosso código.

Informações Úteis do DataFrame

```
df_relatorio.info() #Informações úteis do DataFrame
```

Nesta célula, pegaremos as informações do nosso DataFrame, informações úteis, como Data Type e as colunas do nosso código, sendo essa informação muito importante, pois alguns documentos, as colunas têm nomes diferentes, o que afeta no funcionamento do código, mas sabendo o nome de cada coluna, podemos facilmente e rapidamente consertar este problema. A saída esperada dessa célula quando rodamos é a seguinte:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 762016 entries, 0 to 762015
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Data de abertura      762016 non-null object  
1   Canal                 762016 non-null object  
2   Tema                 762016 non-null object  
3   Assunto              762016 non-null object  
4   Serviço              762016 non-null object  
5   Logradouro           476088 non-null object  
6   Numero               455991 non-null float64  
7   CEP                  455991 non-null object  
8   Setor                0 non-null     float64  
9   Quadra               0 non-null     float64  
10  Latitude             455985 non-null float64  
11  Longitude            455985 non-null float64  
12  Data do Parecer       762016 non-null object  
13  Status da solicitação 762016 non-null object  
14  Orgão                762016 non-null object  
15  Data                 9873 non-null  object  
16  Nível                9846 non-null  object  
17  Prazo Atendimento    9846 non-null  object  
18  Qualidade Atendimento 9843 non-null  object  
19  Atendeu Solicitação  9849 non-null  object  
dtypes: float64(5), object(15)
memory usage: 116.3+ MB
```

Na quarta célula do DataFrame, pegaremos ainda mais informações, e veremos a saída desse DataFrame.

```
pd.DataFrame(df_relatorio) #Saída do DF para ser analisado os nomes das colunas e outras informações necessárias
```

A saída esperada para essa célula é a seguinte:

	dt_cadastro	Canal	Tema	Assunto	Serviço	Logradouro	Número	Cep	Subprefeitura	Distrito	...	Longitude	Status da solicitação	Data do parecer	Órgão	Data de avaliação	sat
0	01/09/2023 00:01	PORTAL ONLINE	Trânsito e Transporte	Estacionamento	Cartão de estacionamento Defis (pessoa com def...		NaN	NaN	NaN	NaN	...	NaN	FINALIZADA	04/09/2023 08:15	SMT-DSV	NaN	pre
1	01/09/2023 00:03	CENTRAL TELEFÔNICA	Lixo e limpeza	Coleta de lixo domiciliar	Reclamar sobre coleta de lixo domiciliar	Rua Catumbi	NaN	NaN	NaN	NaN	...	NaN	FINALIZADA	05/09/2023 05:52	AMLURB	NaN	
2	01/09/2023 00:04	CENTRAL TELEFÔNICA	Urgência	Guarda Civil Metropolitana	Apoio a agente público	Avenida Interlagos	295.0	4660000.0	CIDADE ADEMAR	CIDADE ADEMAR	...	-466.795.604	FINALIZADA	01/09/2023 02:41	GCM	NaN	
3	01/09/2023 00:05	CENTRAL TELEFÔNICA	Rua e bairro	Poluição Sonora - PSIU	Reclamação de poluição	Rua Caurásica	110.0	3813240.0	SAO MIGUEL	VILA JACUI	...	-464.674.586	ABERTA	NaN	SMSUB	NaN	

Com todas essas informações retiradas do DataFrame e definição de variáveis, podemos começar a trabalhar na manipulação dos dados. Como primeiro passo, iremos para a quinta célula do nosso código.

```
df_relatorio.shape #Saída da quantidade de linhas e colunas no DF
```

Nesta célula, pegaremos as dimensões do nosso DataFrame, o que nos mostra a quantidade de linhas e colunas. A saída esperada é a seguinte:

```
(667618, 23)
```

Na sexta célula do nosso Script, recuperaremos a informação de quantas solicitações foram recebidas no período.

```
print(f'O total de Solicitações recebidas no {periodo_ano} de {ano_vigente} foi de {df_relatorio.shape[0]} solicitações recebidas') #Mostrando o total de solicitações recebidas no período
✓ 0.0s
```

Fazemos um print com as variáveis periodo_ano e ano_vigente, definidas lá no começo, e com a função shape[0], mostramos a quantidade de linhas no nosso DataFrame, o que equivale a solicitações recebidas. A saída esperada é essa:

```
O total de Solicitações recebidas no 3º Quadrimestre de 2023 foi de 667618 solicitações recebidas
```

Para a sétima célula, definiremos uma variável que pegará o campo 'Status da Solicitação', e na oitava célula usaremos a função unique(), para mostra individualmente todos os tipos de Status da Solicitação do nosso código.

```
tp_solicitacao = df_relatorio['Status da solicitação'] #Definindo uma variável para o campo Status Solicitação
✓ 0.0s
```

```
tp_solicitacao.unique() #Mostrando todos tipos de Status de solicitação
```

A saída esperada para a oitava célula é a seguinte:

```
array(['FINALIZADA', 'ABERTA', 'CANCELADA', 'EM ANDAMENTO'], dtype=object)
```

Na nona célula, definimos mais uma variável, desta vez definimos ela como 'FINALIZADA', pois queremos pegar apenas as solicitações que têm seu Status como finalizado.

```
solicitacoes_concluidas=['FINALIZADA'] #Definindo Solicitações Concluídas como FINALIZADA
```

Após definir essa variável, na décima célula, faremos um cálculo para verificar quantas solicitações foram concluídas no período analisado.

```
qtd_concluidas = tp_solicitacao.isin(solicitacoes_concluidas).sum() #Calculo para analisar o total de solicitações concluídas  
print(f'A quantidade de solicitações concluídas: {qtd_concluidas}')
```

Na primeira linha usamos a função `isin()` para ver quantos itens da variável `tp_solicitacao` atendem aos critérios da variável `solicitacoes_concluidas`. Após essa verificação ele utiliza a função `sum()`, para somar essas variáveis e mostrar o valor total.

A saída esperada ao executar o código é a seguinte:

```
A quantidade de solicitações concluídas: 612230
```

Para a décima primeira célula do código faremos um processo parecido para verificar o valor total de solicitações concluídas e depois faremos um cálculo para ver o quanto isso equivale em porcentagem.

```
#Verifica quais linhas tem o status FINALIZADA e conta a quantidade de linhas do filtro  
filtro_concluidas = tp_solicitacao.isin(solicitacoes_concluidas)  
total_concluidas = filtro_concluidas.sum()  
  
#Conta o número total de solicitações  
total_solicitacoes = len(df_relatorio)  
  
#Calcula a porcentagem de solicitações concluídas  
porcentagem_concluidas = (total_concluidas/total_solicitacoes) *100  
  
print(f"Quantidade de solicitações concluídas: {total_concluidas}")  
print(f"Porcentagem de solicitações concluídas: {porcentagem_concluidas:.2f}%")
```

Nas duas primeiras linhas, fazemos uma filtragem para que nos traga apenas as solicitações que foram concluídas e depois adicionamos em outra variável com o valor da soma. Para o filtro usamos a função `isin()` e para a soma a função `sum()`.

Após isso, utilizamos a função `len()`, que conta a quantidade total de solicitações no relatório, e armazena na variável `total_solicitacoes`.

Dentro da variável `porcentagem_concluidas`, faremos nosso cálculo de porcentagem. Dividindo o total de solicitações concluídas, pela quantidade total e multiplicando por 100. Após isso, temos a saída dos resultados, que deve ser essa:

```
Quantidade de solicitações concluídas: 612230
Porcentagem de solicitações concluídas: 91.70%
```

Agora iremos analisar as solicitações dos Serviços. Na décima segunda célula definimos uma variável que recebe o campo Serviço do DataFrame e em seguida, utilizando a função `unique()`, mostra todos os tipos de serviços do DataFrame.

```
tp_servicos = df_relatorio['Serviço'] #Definindo uma variável para o campo de Serviços
tp_servicos.unique()#Mostrando todos os tipos de serviços
✓ 0.0s
```

A saída esperada irá mostrar todos os tipos de serviços presentes no DF:

```
array(['Cartão de estacionamento DeFis (pessoa com deficiência)',
      'Reclamar sobre coleta de lixo domiciliar',
      'Apoio a agente público', 'Reclamação de poluição sonora - PSIU',
      'Baixas temperaturas ? Solicitar atendimento social',
      'Ponto de ônibus ? Solicitar manutenção, conserto e/ou limpeza de equipamento existente',
      'Lixo domiciliar ? Denunciar colocação de lixo fora do horário de coleta',
      'Farmácias de manipulação e drogarias - Entregar Balanços e Relações Mensais de Notificação de Receitas da Portaria 344/98',
      'Reparação de danos - Solicitar indenização por danos materiais',
      'Denunciar invasão de áreas municipais',
      'Ocorrências com animais silvestres',
      'Ônibus - Reclamar de descumprimento de partida no ponto inicial/final',
      'Programa Bolsa Família - Consultar situação do benefício',
      'Cartão de estacionamento idoso',
      'Ônibus - Reclamar de intervalo excessivo da linha',
      'Entulho e grandes objetos em via pública - solicitar remoção',
      'Ônibus - Elogiar o(a) motorista, cobrador(a) ou fiscal',
      'Intensificar fiscalização de veículos estacionados/parados irregularmente',
      'Programa Cidade Solidária - Regularizar situação',
      'Ônibus - Denunciar motorista por não atender ao sinal de embarque e desembarque',
      'Ônibus - Denunciar veículo clandestino ou não credenciado',
      'Ocorrências com Drogas',
      'Estações de bicicletas compartilhadas - solicitar mudança de local',
      'Tapa-buraco',
      'Ônibus - Reclamar de motorista por atraso na partida',
      'Farmácias de manipulação e drogarias - Entregar Receitas de Medicamentos de Controle Especial Emergenciais ou que sejam de outros estados',
      ...
      'CIPTEA ? Alterar dados e/ou Renovar validade',
      'SPTrans - Suporte para instituição de ensino',
      'Ônibus fretado - Reclamação da conservação e manutenção do veículo',
      'Descomplica SP Campo Limpo - Sugestão',
      'Denunciar violência e discriminação contra jovens'], dtype=object)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Agora na décima terceira célula para fins de informação, vemos a quantidade de serviços no DF, utilizando a função `unique()` e `shape()` para verificar essa informação.

```
tp_servicos.unique().shape[0]#Mostra a quantidade total de serviços
```

A saída esperada é a seguinte:

```
642
```

Na décima quarta célula do código, faremos a contagem da frequência de cada serviço no DF e mostramos o Top 10 serviços mais solicitados.

```
#Conta a frequência de cada serviço no campo Serviço
contagem_servicos = df_relatorio['Serviço'].value_counts()

#Seleciona os top 10 serviços mais solicitados
top_10_servicos = contagem_servicos.head(10)

print("Top 10 Serviços mais Solicitados")
print(top_10_servicos)
```

Na primeira linha, usamos a função `value_counts()` para verificar a frequência que cada item de Serviço aparece no DF, em seguida, armazenamos essa informação na variável `contagem_servicos`.

Na segunda linha, criamos uma variável chamada `top_10_servicos`, e com a auxílio da função `head()`, verificamos os 10 serviços com maior frequência. Nas duas últimas linhas, fazemos o uso da função `print()` para imprimir a saída gerada. A saída esperada é essa:

```
Top 10 Serviços mais Solicitados
Serviço
Baixas temperaturas ? Solicitar atendimento social      76677
Cartão de estacionamento idoso                          47314
Tapa-buraco                                              42395
Entulho e grandes objetos em via pública - solicitar remoção 38652
Árvore ? Solicitar avaliação em calçadas e praças para fins de poda ou remoção 26217
Centros esportivos ? Solicitar carteirinha              24445
Rodízio de veículos - Isenção para pessoa com deficiência (PCD) 18791
Programa Bolsa Família - Consultar situação do benefício 16245
Senha Web - Solicitar desbloqueio para Pessoa Física, MEI e Empresário Individual 15840
Reclamação de poluição sonora - PSIU                    15566
Name: count, dtype: int64
```

Para a décima quinta célula, veremos a porcentagem de solicitações de cada um desses serviços do top 10 e quanto equivale a soma deles em relação a outros serviços.

```
#Calcula a porcentagem de cada um dos serviços do top 10
top_10_servicos_porcentagem = (top_10_servicos / total_solicitacoes) * 100

#Formata para 2 casas decimais
top_10_servicos_porcentagem = top_10_servicos_porcentagem.round(2)

#Soma a conclusão das porcentagens dos top 10 serviços
soma_porcentagem_top_10 = top_10_servicos_porcentagem.sum()

print("Top 10 serviços mais solicitados com porcentagem")
print(top_10_servicos_porcentagem)
print(f"Soma das porcentagens dos top 10 serviços: {soma_porcentagem_top_10:.2f}%")
```

✓ 0.0s

Na primeira linha de código, fazemos o cálculo para verificar a porcentagem dos itens do top 10 e armazenamos na variável `top_10_servicos_porcentagem`, pegando os serviços, dividindo pelo total de solicitações e dividindo por 100. Já na segunda linha, fazemos a formatação da

variável ajustando para mostrar o valor em apenas 2 casas decimais, para isso, temos o auxílio da função `round()`.

Na terceira linha, fazemos a soma de todas essas porcentagens com a função `sum()` e armazenamos na variável `soma_porcentagem_top_10`. Nas últimas linhas da célula imprimimos esses valores com a função `print()`. A saída esperada ao rodar o código é a seguinte:

```
Top 10 servicos mais solicitados com porcentagem
Serviço
Baixas temperaturas ? Solicitar atendimento social      11.49
Cartão de estacionamento idoso                          7.09
Tapa-buraco                                             6.35
Entulho e grandes objetos em via pública - solicitar remoção 5.79
Árvore ? Solicitar avaliação em calçadas e praças para fins de poda ou remoção 3.93
Centros esportivos ? Solicitar carteirinha              3.66
Rodízio de veículos - Isenção para pessoa com deficiência (PCD) 2.81
Programa Bolsa Família - Consultar situação do benefício 2.43
Senha Web - Solicitar desbloqueio para Pessoa Física, MEI e Empresário Individual 2.37
Reclamação de poluição sonora - PSIU                  2.33
Name: count, dtype: float64
Soma das porcentagens dos top 10 serviços: 48.25%
```

Agora faremos um processo parecido, só que dessa vez com os Órgãos.

```
tp_orgao = df_relatorio['Órgão'] #Definindo uma variável para o campo Órgão
tp_orgao.unique() #Mostra todos os tipos de Órgão
```

Na décima sexta célula, armazenamos todos os itens da coluna Órgão na variável `tp_orgao` e depois com o auxílio da função `unique()`, ele nos retorna todos os órgãos do DF. A saída esperada é a seguinte:

```
array(['SMT-DSV', 'AMLURB', 'GCM', 'SMSUB', 'SMADS', 'SPOBRAS', 'SMS',
      'PGM', 'SPTRANS', 'CET', '#N/D', 'SMT-DTP', 'COVISA-UVIS', 'SF',
      'SMS-COSAP', 'OGM', 'COVISA-DVZ', 'SMSU/Defesa Civil', 'SMIT',
      'SME', 'COVISA-DVPSIS', 'SMPED', 'SEME', 'SMSU', 'SMSO', 'SVMA',
      'SMJ-Procon', 'SMDHC', 'SMIT - Descomplica SP', 'SEL', 'SMC',
      'SMDet', 'SEHAB', 'INTEGRACAO-SEI', 'COHAB', '.', 'SGM', 'SMRI',
      'CETEL', 'SPCINE', 'SMPDE'], dtype=object)
```

Na décima sétima célula do código, iremos verificar quais são os top 5 órgãos mais requisitados e o quanto isso equivale em porcentagem para cada um deles.

```

#Conta a frequência de cada órgão no campo Órgão
orgaos_contagem = tp_orgao.value_counts()

#Seleciona os top 5 órgãos mais frequentes
top_5_orgaos = orgaos_contagem.head(5)

#Calcula as porcentagens de cada um dos órgãos
porcentagem_orgaos = (top_5_orgaos / total_solicitacoes) * 100

#Formata para 2 casas decimais
porcentagem_orgaos = porcentagem_orgaos.round(2)

#Soma as porcentagens dos top 5 orgaos
soma_porcentagem_orgaos = porcentagem_orgaos.sum()

print("Top 5 Órgãos mais frequentes com porcentagens:")
print(porcentagem_orgaos)
print(f"Soma das porcentagens dos top 5 Órgãos: {soma_porcentagem_orgaos:.2f}%")

```

Na primeira linha, contaremos a frequência de cada um dos órgãos na coluna órgãos do DF, para isso, usaremos a função `value_counts()` e armazenaremos esse valor na variável `orgaos_contagem`. Logo em seguida, utilizaremos a função `head()`, para identificar os top 5 órgãos mais requisitados e armazenaremos na variável `top_5_orgaos`. Com isso definido, iremos para a terceira linha do código, onde na variável `porcentagem_orgaos`, fazemos os cálculos de porcentagem, pegando a variável `top_5_orgaos`, dividindo-a pela `total_solicitacoes` e multiplicando por 100.

Na quarta linha, utilizamos a função `round()` para formatar para 2 casas decimais. Na quinta linha, fazemos a soma dessas porcentagens, e armazenamos na variável `soma_porcentagem_orgaos`, para fazer a soma, utilizamos o auxílio da função `sum()`. Nas últimas linhas, utilizamos a função `print()` para imprimir os resultados. A saída esperada é essa:

```

Top 5 Órgãos mais frequentes com porcentagens:
Órgão
SMSUB      21.09
SMADS      15.43
SMT-DSV    11.97
AMLURB     11.01
SF          6.01
Name: count, dtype: float64
Soma das porcentagens dos top 5 Órgãos: 65.51%

```

Agora, na décima oitava célula, faremos o total de solicitações concluídas dentro dos top 5 órgãos mais requisitados, e mostraremos esse valor em porcentagem.

```
#Filtrar o dataframe para incluir apenas solicitações dos top 5 órgãos
df_top_5_orgaos = df_relatorio[df_relatorio['Órgão'].isin(top_5_orgaos.index)]

#Filtrar dataframe para incluir apenas as solicitações concluídas dentro dos top 5 órgãos
df_concluidas_top_5 = df_top_5_orgaos[filtro_concluidas]

#Contar a quantidade de solicitações concluídas e total dentro dos top 5 órgãos
qtd_concluidas_top_5 = len(df_concluidas_top_5)
total_top_5_orgaos = len(df_top_5_orgaos)

#Calcular a porcentagem de solicitações concluídas dentro dos top 5 órgãos
porcentagem_orgaos_top_5 = (qtd_concluidas_top_5 / total_top_5_orgaos) * 100

print(f"Quantidade de solicitações concluídas dentro dos top 5 órgãos: {qtd_concluidas_top_5}")
print(f"Porcentagem de solicitações concluídas dentro dos top 5 órgãos: {porcentagem_orgaos_top_5:.2f}%")
```

✓ 0.1s

Na primeira linha, usaremos a função `isin()`, para filtrar e incluir apenas as solicitações dos top 5 órgãos no DF, armazenando este valor na variável `df_top_5_orgaos`.

Para a segunda linha, filtramos apenas as solicitações concluídas desta variável criada, armazenando em outra variável, a `df_concluidas_top_5`.

Para a terceira e quarta linha, usamos a função `len()` para contar a quantidade de solicitações concluídas e total dentro do nosso top 5, e armazenamos respectivamente nas variáveis `qtd_concluidas_top_5` e `total_top_5_orgaos`.

Na quinta linha da célula, iremos fazer o cálculo da porcentagem, dividindo `qtd_concluidas_top_5` por `total_top_5_orgaos` e multiplicando por 100.

Nas duas últimas linhas, aplicamos a função `print()` para imprimir o valor. A saída esperada é essa:

```
Quantidade de solicitações concluídas dentro dos top 5 órgãos: 395039
Porcentagem de solicitações concluídas dentro dos top 5 órgãos: 90.32%
```

Agora, para o próximo passo do nosso Script, iremos fazer a verificação do uso de cada Canal.

Como primeiro passo, definiremos uma variável `tp_canais`, que receberá todos os canais do DF e com o auxílio da função `unique()` veremos todos os canais do DF

```
tp_canais = df_relatorio['Canal']
tp_canais.unique()
```

✓ 0.0s

```
array(['PORTAL ONLINE', 'CENTRAL TELEFÔNICA', 'APLICATIVO', 'CHATBOT',
      'EMAIL'], dtype=object)
```

Na vigésima célula do nosso código, iremos verificar o total de solicitações recebidas e finalizadas em cada canal e veremos o quanto isso equivale em porcentagem para cada um desses canais.

```
#Contar a quantidade de solicitações recebidas por canal
solicitacoes_por_canal = df_relatorio['Canal'].value_counts()

#Contar a quantidade de solicitações concluídas em cada Canal
solicitacoes_concluidas_por_canal = df_relatorio[df_relatorio['Status da solicitação'].isin(['FINALIZADA'])]['Canal'].value_counts()

#Criar um DataFrame para armazenar os resultados
resultados_canais = pd.DataFrame({
    'Total Solicitações': solicitacoes_por_canal,
    'Solicitações Concluídas': solicitacoes_concluidas_por_canal
}).fillna(0).astype(int)

#Calcular a porcentagem de solicitações finalizadas em cada canal
resultados_canais['Porcentagem Finalizadas'] = (resultados_canais['Solicitações Concluídas'] / resultados_canais['Total Solicitações']) * 100

#Calcular a porcentagem de solicitações recebidas por canal
resultados_canais['Porcentagem Recebidas'] = (resultados_canais['Total Solicitações'] / len(df_relatorio)) * 100

#Formatar a porcentagem para 2 casas decimais
resultados_canais = resultados_canais.round(2)

print("Resultados por Canal de Atendimento")
print(resultados_canais)
```

Na primeira linha do código, utilizamos a função `value_counts()`, para contar quantas solicitações foram recebidas por canal.

Na segunda linha, utilizando as funções `isin()` e `value_counts()`, contamos e filtramos as solicitações que foram finalizadas, armazenando isso na variável `solicitacoes_concluidas_por_canal`.

Já na terceira linha do código, criamos a variável `resultados_canais`. Nesta variável, criaremos um DF, e nele colocaremos o total de solicitações por canais, chamando a variável `solicitacoes_por_canal` e a variável `solicitacoes_concluidas_por_canal`, para nos mostrar o total de solicitações concluídas em cada um dos canais de atendimento. Utilizando a função `fillna()`, transformamos todos os resultados nulos em 0 e em seguida transformamos todas essas informações criadas em int, usando a função `astype()`.

Nas últimas linhas do código, fazemos os cálculos de porcentagem necessários, e com a função `print()` imprimimos o resultado. A saída esperada é essa:

```

.. Resultados por Canal de Atendimento

```

	Total Solicitações	Solicitações Concluídas \
Canal		
PORTAL ONLINE	360346	328356
CENTRAL TELEFÔNICA	252443	232426
APLICATIVO	40243	38574
CHATBOT	14581	12873
EMAIL	5	1

	Porcentagem Finalizadas	Porcentagem Recebidas
Canal		
PORTAL ONLINE	91.12	53.97
CENTRAL TELEFÔNICA	92.07	37.81
APLICATIVO	95.85	6.03
CHATBOT	88.29	2.18
EMAIL	20.00	0.00

Agora para a finalização do código, na vigésima primeira célula, criaremos o nosso Excel com todos os dados que retiramos.

```

# Instale o xlsxwriter, se ainda não estiver instalado
!pip install xlsxwriter

# Dados para o Excel
dados = {
    'Resumo Geral': {
        'Total de Solicitações': [df_relatorio.shape[0]],
        'Quantidade de Solicitações Concluídas': [qtd_concluidas],
        'Porcentagem de Solicitações Concluídas': [f'{(qtd_concluidas / df_relatorio.shape[0]) * 100:.2f}%']
    },
    'Top 10 Serviços': {
        'Serviço': top_10_servicos.index,
        'Quantidade': top_10_servicos.values
    },
    'Top 10 Serviços %': {
        'Serviço': top_10_servicos.index,
        'Quantidade': top_10_servicos.values,
        'Porcentagem': top_10_servicos_porcentagem.values
    },
    'Top 5 Órgãos %': {
        'Órgão': porcentagem_orgaos.index,
        'Porcentagem': porcentagem_orgaos.values
    },
    'Concluídas nos Top 5 Órgãos': {
        'Órgão': [f'Quantidade: {qtd_concluidas_top_5}', f'Porcentagem: {porcentagem_orgaos_top_5:.2f}%']
    },
    'Resultados por Canal': resultados_canais.reset_index()
}

# Criar um escritor de Excel
with pd.ExcelWriter(nome_arquivo, engine='xlsxwriter') as writer:
    # Adicionar cada DataFrame como uma planilha
    for sheet_name, data in dados.items():
        df = pd.DataFrame(data)
        df.to_excel(writer, sheet_name=sheet_name, index=False)

```

O código começa instalando o pacote xlsxwriter com o comando `!pip install xlsxwriter`. Esse pacote é necessário para salvar dataframes do Pandas em arquivos Excel (.xlsx).

Em seguida, é criado um dicionário chamado `dados` que contém diferentes seções com os dados que serão escritos em diferentes planilhas do arquivo Excel. Cada chave do dicionário representa o nome de uma planilha, e o valor associado é outro dicionário ou dataframe contendo os dados a serem escritos nessa planilha.

- Na planilha "Resumo Geral", os dados são:
 - Total de Solicitações: o total de registros no dataframe `df_relatorio` usando `df_relatorio.shape[0]`.
 - Quantidade de Solicitações Concluídas: a quantidade de solicitações concluídas armazenadas na variável `qtd_concluidas`.
 - Porcentagem de Solicitações Concluídas: a porcentagem de solicitações concluídas, formatada para duas casas decimais usando `f"{porcentagem_concluidas:.2f}%"`.
- Na planilha "Top 10 Serviços", os dados são:
 - Serviço: os índices de `top_10_servicos`.
 - Quantidade: os valores de `top_10_servicos`.
- Na planilha "Top 10 Serviços %", os dados são:
 - Serviço: os índices de `top_10_servicos`.
 - Quantidade: os valores de `top_10_servicos`.
 - Porcentagem: as porcentagens de `top_10_servicos_porcentagem`.
- Na planilha "Top 5 Órgãos %", os dados são:
 - Órgão: os índices de `porcentagem_orgaos`.
 - Porcentagem: os valores de `porcentagem_orgaos`.
- Na planilha "Concluídas nos Top 5 Órgãos", os dados são:
 - Órgão: uma lista com duas strings, uma indicando a quantidade de solicitações concluídas (`qtd_concluidas_top_5`) e outra indicando a porcentagem de solicitações concluídas (`porcentagem_orgaos_top_5`).
- Na planilha "Resultados por Canal", os dados são:
 - Um dataframe resultante da redefinição dos índices do dataframe `resultados_canais` com `resultados_canais.reset_index()`.

O código então cria um escritor de Excel com o `pd.ExcelWriter(nome_arquivo, engine='xlsxwriter')`. Este escritor é usado para salvar os dataframes no arquivo Excel especificado por `nome_arquivo`.

Dentro do contexto do escritor (with statement), o código itera sobre o dicionário `dados`. Para cada par `sheet_name, data`:

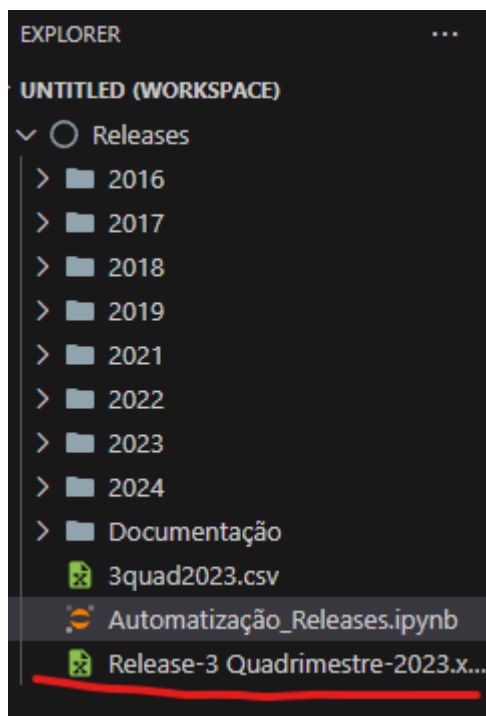
- Um dataframe `df` é criado a partir de `data`.
- Esse dataframe `df` é escrito em uma planilha do arquivo Excel com o nome especificado por `sheet_name` usando `df.to_excel(writer, sheet_name=sheet_name, index=False)`. O argumento `index=False` é usado para não incluir os índices dos dataframes como uma coluna adicional no Excel.

Finalmente, ao sair do contexto with, o arquivo Excel é salvo automaticamente com todas as planilhas e dados especificados.

Ao finalizar o código, a saída esperada é essa:

```
Defaulting to user installation because normal site-packages is not writeable  
Requirement already satisfied: xlswriter in c:\users\x580594\appdata\roaming\python\python312\site-packages (3.2.0)
```

Quando finalizado, o arquivo Excel gerado irá aparecer na mesma pasta que está seu código.



Nome	Data de modificação	Tipo	Tamanho
2016	30/07/2024 14:07	Pasta de arquivos	
2017	30/07/2024 14:06	Pasta de arquivos	
2018	30/07/2024 14:06	Pasta de arquivos	
2019	30/07/2024 14:27	Pasta de arquivos	
2021	30/07/2024 14:38	Pasta de arquivos	
2022	30/07/2024 15:00	Pasta de arquivos	
2023	31/07/2024 10:28	Pasta de arquivos	
2024	31/07/2024 10:34	Pasta de arquivos	
Documentação	31/07/2024 11:45	Pasta de arquivos	
3quad2023.csv	31/07/2024 14:21	Arquivo de Valore...	145.737 KB
Automatização_Releases.ipynb	01/08/2024 10:33	Arquivo Fonte Jup...	105 KB
Release-3 Quadrimestre-2023.xlsx	01/08/2024 09:44	Planilha do Micro...	9 KB

Observações

- **Uso do `xlsxwriter`:** É necessário garantir que o módulo `xlsxwriter` esteja instalado no ambiente onde o script será executado. Este módulo é utilizado pelo `pandas` para criar e manipular arquivos Excel. O comando `!pip install xlsxwriter` deve ser executado antes de rodar o script.
- **Compatibilidade com `Pandas`:** O `pandas` deve estar atualizado para garantir a compatibilidade com o `xlsxwriter` e evitar problemas na escrita dos arquivos Excel. Recomenda-se utilizar versões recentes de ambos os pacotes.
- **Formatos de Dados:** Os dados são organizados em dicionários, onde as chaves representam os nomes das planilhas e os valores são os dados correspondentes. É importante garantir que os dados estejam estruturados corretamente antes de serem convertidos em `DataFrames`. Verifique se os dados estão no formato correto (listas, séries ou `dataframes`) para evitar erros de conversão.
- **Uso de `DataFrames`:** O código utiliza `pandas.DataFrame` para transformar os dados em tabelas que serão escritas no arquivo Excel. Cada `DataFrame` é escrito em uma

planilha separada, conforme especificado no dicionário dados. Este método permite flexibilidade na organização dos dados e facilita a manipulação e análise subsequente.

- **Cuidado com os Índices:** Ao converter os dados em DataFrames, os índices dos dados originais são preservados. Se necessário, os índices podem ser redefinidos utilizando o método `reset_index()`. Isso é especialmente útil quando se deseja que os índices não sejam incluídos como colunas no arquivo Excel.
- **Formato das Porcentagens:** As porcentagens são formatadas como strings com duas casas decimais e um símbolo de porcentagem (%). É importante manter este formato para garantir a consistência e a legibilidade dos dados no arquivo Excel. Verifique se todas as porcentagens estão corretamente calculadas e formatadas antes da escrita.
- **Nomes das Planilhas:** Os nomes das planilhas são definidos pelas chaves do dicionário dados. É importante escolher nomes descritivos e curtos para facilitar a navegação no arquivo Excel. Evite caracteres especiais e mantenha os nomes dentro do limite de 31 caracteres imposto pelo Excel.
- **Gerenciamento do Writer:** O uso do `with pd.ExcelWriter(nome_arquivo, engine='xlsxwriter')` garante que o escritor de Excel seja corretamente gerenciado, fechando o arquivo adequadamente após a escrita de todas as planilhas. Isso evita corrupção de arquivos e garante que todas as operações de escrita sejam concluídas corretamente.
- **Validação dos Dados:** Antes de escrever os dados no arquivo Excel, é recomendável realizar validações para garantir que todos os valores estejam corretos e dentro do esperado. Isso inclui checar se todas as colunas esperadas estão presentes e se os valores não contêm erros ou inconsistências.
- **Manutenção do Código:** É importante manter o código atualizado e bem documentado para facilitar futuras manutenções. Comentários no código podem ajudar a explicar a lógica por trás de certas operações e a estrutura dos dados.
- **Testes:** Realize testes para verificar se o arquivo Excel gerado está conforme o esperado. Isso inclui verificar se todas as planilhas foram criadas corretamente, se os dados estão completos e formatados corretamente, e se o arquivo pode ser aberto sem problemas.
- **Desempenho:** Para grandes volumes de dados, a operação de escrita no Excel pode ser demorada. Considere otimizações se necessário, como a divisão do processo em etapas ou a utilização de hardware mais potente.
- **Backup de Dados:** Sempre mantenha cópias de backup dos dados antes de realizar operações de escrita. Isso previne perda de dados em caso de falhas durante a execução do script.

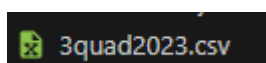
Erros Possíveis e Soluções

Nome do Arquivo

Esse é o erro mais comum que pode acontecer, digitação do nome errado do arquivo de importação. Ao executar o código, a primeira coisa que ele irá solicitar, é o nome do arquivo que você está digitando, para isso, verifique se o nome do arquivo está correto antes de confirmar a ação

Ex:

Nome do arquivo que estou importando:



Exemplo de entrada errada:

3quadrimestre2023

Digite o nome do arquivo que você irá importar: (Press 'Enter' to confirm or 'Escape' to cancel)

definição de Variáveis fixas e importação do arquivo .csv > importacao = input("Digite o nome do arquivo q

Saída que será gerada na segunda célula caso o nome do arquivo esteja errado:

```
-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[25], line 3
      1 import pandas as pd #Import do Pandas
----> 3 df_relatorio = pd.read_csv(importacao, sep=';', encoding='ISO-8859-1')# Transformando o arquivo importado em dataframe

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, f
    1013 kws_defaults = _refine_defaults_read(
    1014     dialect,
    1015     delimiter,
    (...)
    1022     dtype_backend=dtype_backend,
    1023 )
    1024 kws.update(kws_defaults)
-> 1026 return _read(filepath_or_buffer, kws)

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:620, in _read(filepath_or_buffer, kws)
    617 _validate_names(kws.get("names", None))
    619 # Create the parser.
-> 620 parser = TextFileReader(filepath_or_buffer, **kws)
    622 if chunksize or iterator:
    623     return parser

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.__init__(self, f, engine, **kws)
    1617     self.options["has_index_names"] = kws["has_index_names"]
    ...
    880     else:
    881         # Binary mode
    882         handle = open(handle, ioargs.mode)

FileNotFoundError: [Errno 2] No such file or directory: '3quadrimestre2023.csv'
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

A maneira certa é importando o nome do arquivo da forma correta. Ex:

Nome do arquivo:

3quad2023.csv

Entrada do usuário:

3quad2023

Digite o nome do arquivo que você irá importar: (Press 'Enter' to confirm or 'Escape' to cancel)

inição de Variáveis fixas e importação do arquivo .csv > importacao = input("Digite o nome do arquivo q

Saída resultante na segunda linha:

```
import pandas as pd #Import do Pandas

df_relatorio = pd.read_csv(importacao, sep=';', encoding='ISO-8859-1')# Transformando o arquivo importado em dataframe
✓ 14s

C:\Users\X580594\AppData\Local\Temp\ipykernel_24212\875109203.py:3: DtypeWarning: Columns (6,7) have mixed types. Specify dtype option on import or set low_memory=False.
df_relatorio = pd.read_csv(importacao, sep=';', encoding='ISO-8859-1')# Transformando o arquivo importado em dataframe
```

Erro linguagem de codificação

Outro erro que pode vir a ocorrer é com a linguagem de codificação. Alguns documentos Excel não são compatíveis para leitura por conta do código ASCII. Para isso, foi inserido no código, um termo a mais para a transformação do .csv em DataFrame. Linha 3, Célula 2, o campo “encoding”, foi definido para a linguagem de codificação binária de comprimento “ISO-8859-1”. No caso deste campo dar algum problema, pode ser feito a alteração para “UTF-8”. Caso a UTF-8 também dê problema, use alguma outra linguagem de codificação para contornar o erro.

Exemplo desse erro visualmente:

```
import pandas as pd #Import do Pandas

df_relatorio = pd.read_csv(importacao, sep=';')# Transformando o arquivo importado em dataframe
ⓧ 0.0s

-----
UnicodeDecodeError                                Traceback (most recent call last)
Cell In[28], line 3
      1 import pandas as pd #Import do Pandas
----> 3 df_relatorio = pd.read_csv(importacao, sep=';')# Transformando o arquivo importado em dataframe

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, na
    1013 kwds_defaults = _refine_defaults_read(
    1014     dialect,
    1015     delimiter,
    (...)
    1022     dtype_backend=dtype_backend,
    1023 )
    1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:620, in _read(filepath_or_buffer, kwds)
    617 _validate_names(kwds.get("names", None))
    619 # Create the parser.
-> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
    622 if chunksize or iterator:
    623     return parser

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)
    1617     self.options["has_index_names"] = kwds["has_index_names"]
    ...

File parsers.pyx:2053, in pandas._libs.parsers.raise_parser_error()

File <frozen codecs>:322, in decode(self, input, final)

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 36: invalid continuation byte
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Esse é um erro menos comum, pois é uma parte do código que não deve ser alterada, mas caso venha a ocorrer, você já sabe o que deve ser feito.

Erro de Colunas do DataFrame

Este é um erro mais comum. Pode acontecer caso aconteça alguma mudança na forma como o DataFrame é foi importado. Por exemplo, em algumas colunas do DataFrame, podem ter mudanças nos nomes das colunas, porque a importação é feita a partir de um .csv, mas não se preocupe, é um erro fácil de ser corrigido. Exemplo:

Saída com erro:

```
tp_solicitacao = df_relatorio['Status da solicitação']#Definindo uma variável para o campo Status Solicitação
0.2s

-----
KeyError                                Traceback (most recent call last)
file ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\indexes\base.py:3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

file index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

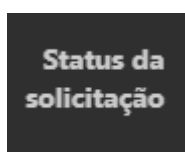
file index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

file pandas\_libs\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.PyObjectHashTable.get_item()

file pandas\_libs\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Status da solicitação'
```

Verificando a forma como o campo está escrito no DF:



Após vermos isso, notamos que no nosso código está escrito “solicitacao” e não “solicitação”. Para corrigir, mudamos o nome do campo para a forma correta:

```
tp_solicitacao = df_relatorio['Status da solicitação']#Definindo uma variável para o campo Status Solicitação
0.0s
```

Código consertado, saída sem problemas.

Erros nas instalações de bibliotecas

Um erro que pode vir a acontecer é a não instalação das bibliotecas necessárias para o uso do código, por isso, verifique se as bibliotecas pandas e xlswriter estão instaladas no seu computador. Caso não estejam instaladas, abra o seu CMD e digite:

```
Microsoft Windows [versão 10.0.22631.3737]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\X580594>pip install pandas|
```

```
Microsoft Windows [versão 10.0.22631.3737]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\X580594>pip install xlswriter|
```

Usando esses comandos, você instalará as bibliotecas. No próprio código existe uma linha para prevenir esse erro com o `xlswriter`.

```
# Instale o xlswriter, se ainda não estiver instalado  
!pip install xlswriter
```

Com todas as bibliotecas instaladas o código deverá rodar normalmente e sem mais problemas.

Links Úteis e Bibliografia

Para o desenvolvimento desse script, foram utilizados alguns sites e documentações úteis e seguros para esse processo.

Os links para esses sites e documentações se encontrarão abaixo:

W3Schools - <https://www.w3schools.com/>

W3Schools Python - https://www.w3schools.com/python/python_getstarted.asp

W3Schools Pandas - https://www.w3schools.com/python/pandas/pandas_getting_started.asp

Pandas Documentation - <https://pandas.pydata.org/docs/>

Python Documentation - <https://docs.python.org/3/>

XLSXWriter Documentation - <https://xlswriter.readthedocs.io/>

Stack Overflow - <https://stackoverflow.com/>

Chat GPT - <https://chatgpt.com/>

Glossário

- **csv (Comma Separated Values):** Formato de arquivo usado para armazenar dados tabulares de forma simples.
- **.ipynb:** Extensão de arquivos para notebooks Jupyter.
- **.xlsx:** Formato de arquivo Excel usado para armazenar dados tabulares com mais funcionalidades que o CSV.
- **DataFrame:** Estrutura de dados bidimensional do Pandas, semelhante a uma tabela, que facilita a manipulação e análise de dados.
- **df['coluna'].isin(lista):** Filtra o DataFrame para incluir apenas as linhas em que a coluna especificada contém valores presentes na lista fornecida.
- **df['coluna'].unique():** Retorna os valores únicos de uma coluna específica em um DataFrame.
- **df['coluna'].value_counts():** Conta a frequência de cada valor único em uma coluna específica.
- **df.info():** Fornece um resumo das informações do DataFrame, incluindo tipos de dados e contagem de valores não nulos.
- **df.shape:** Retorna as dimensões (linhas, colunas) de um DataFrame.
- **df.to_excel('arquivo.xlsx', index=False):** Exporta o DataFrame para um arquivo Excel, sem incluir os índices das linhas.
- **import pandas as pd:** Importa a biblioteca Pandas e a apelida de pd.
- **pd.read_csv():** Função do Pandas para ler arquivos CSV e transformá-los em DataFrames.
- **Pandas:** Biblioteca Python poderosa para manipulação e análise de dados, especialmente útil para operações com DataFrames.
- **Python:** Linguagem de programação utilizada para automatizar o processo de análise e manipulação de dados nos releases de dados abertos.
- **Visual Studio Code (VSCode):** Um editor de código-fonte que inclui suporte para desenvolvimento em diversas linguagens de programação. Utilizado para escrever e executar o código Python.
- **XlsxWriter:** Biblioteca Python usada para escrever arquivos Excel.